Unsupervised Learning

PROG2051

Instructor: Mohib Ullah

09 April 2026

Kristiania

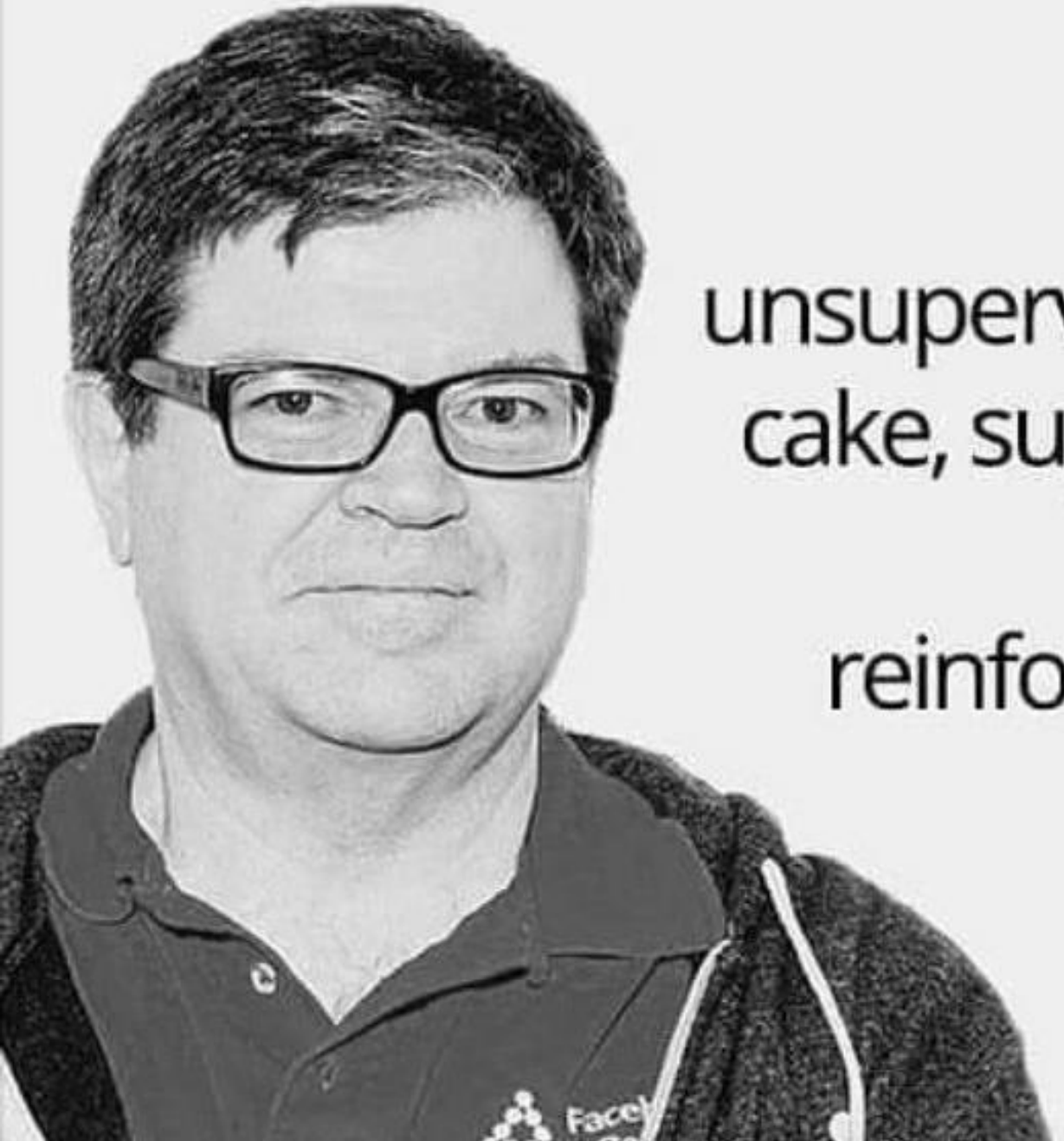
Content

- Unsupervised Learning
- Clustering
- Dimension Reduction

Unsupervised learning

- Unsupervised learning is the class of machine learning algorithm that is used for inference from unlabeled datasets.

- Technically, in unsupervised learning, we want to find some structure in the data and group the datapoints according to some criteria.



"Most of human and animal learning is unsupervised learning. If intelligence was a cake, unsupervised learning would be the cake, supervised learning would be the icing on the cake, and reinforcement learning would be the cherry on the cake.

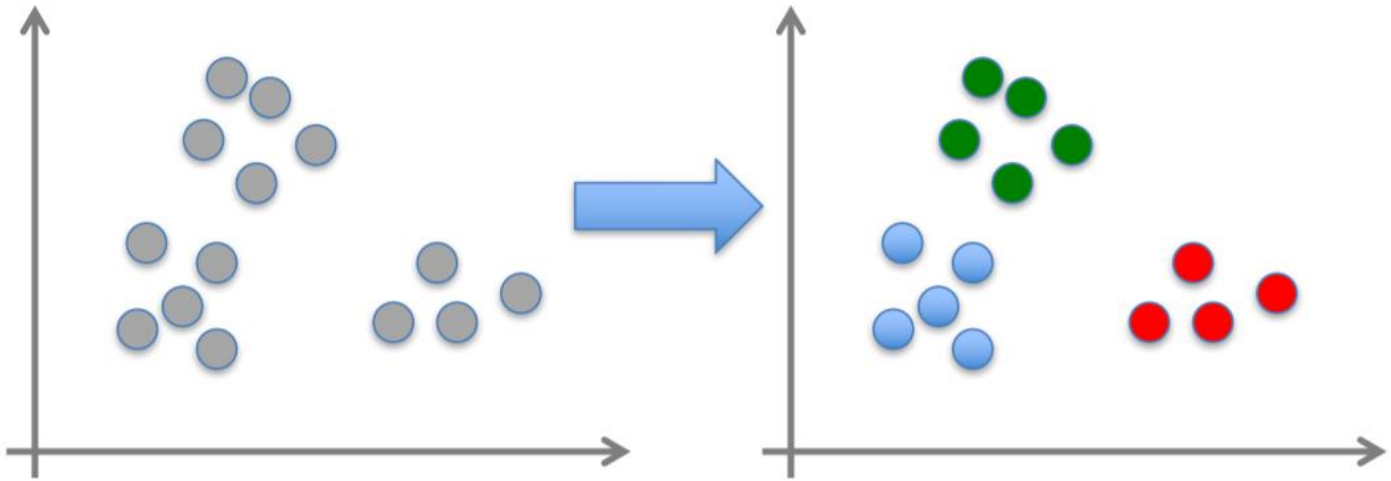
~ Yann Lecun (On true AI)

Carnegie Mellon University

Unsupervised Learning

- Given data $x_1, x_2, \ldots, x_n$ (without labels)

- Cluster the unsorted data according to similarities, patterns and difference in the data points.



Clustering

- Clustering is a technical term that is used to find similarity groups in our data.

- In simple words, clustering groups data instances that are similar to (near) each other in one cluster and data instances that are very different (far away) from each other into different clusters.

Clustering

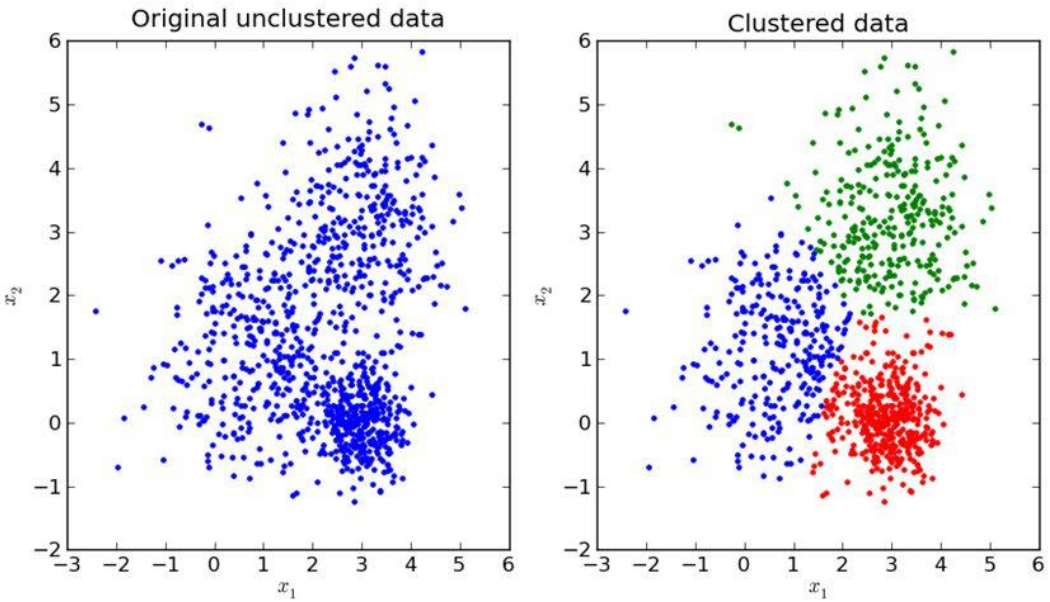
- Due to historical reasons, clustering is often considered synonymous with unsupervised learning.

- One possible reason for this is that in unsupervised learning, there are no class values that can denote a priori grouping of the data instances which is the canonical case of supervised learning.

Applications

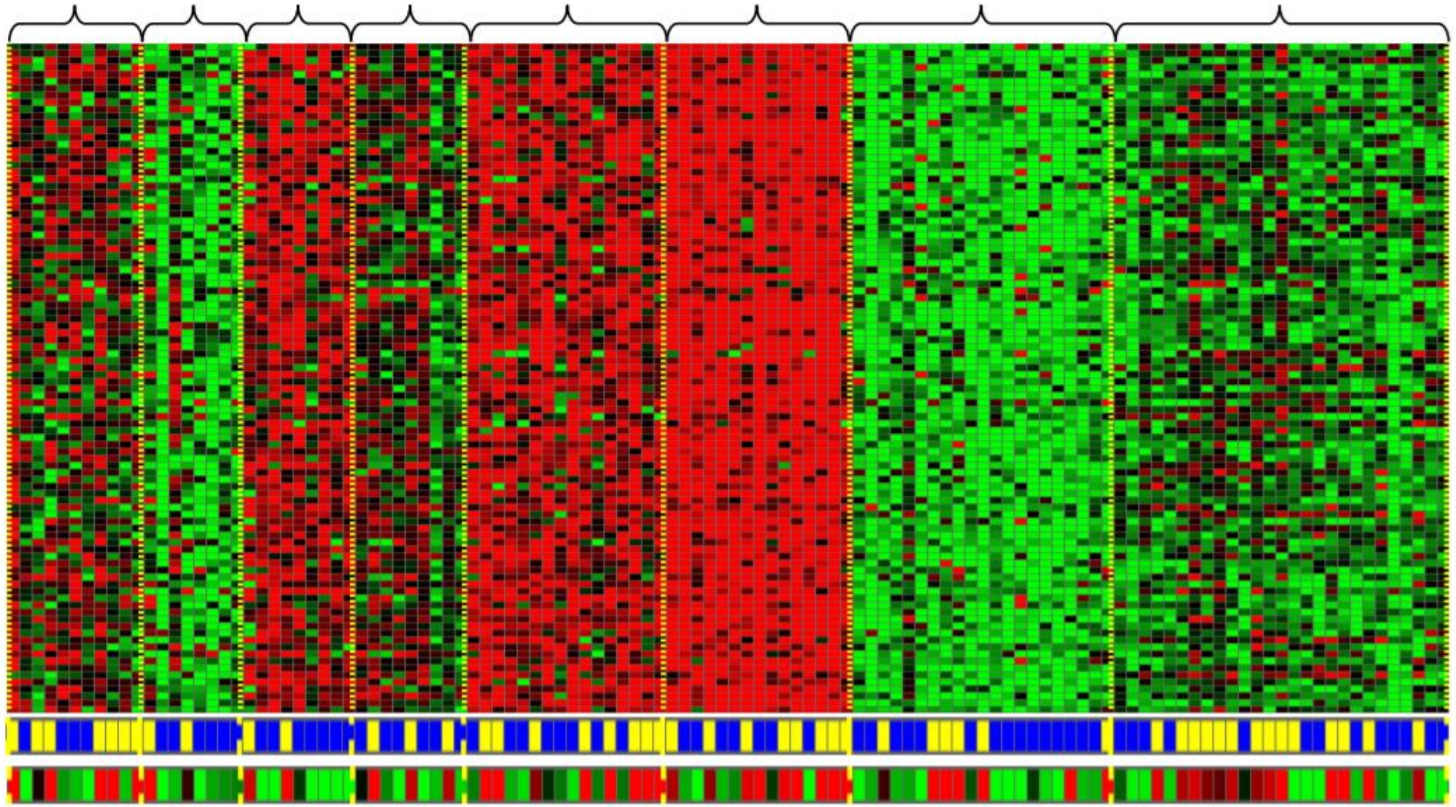
Data Clustering
& Visualization

Unsupervised Learning



Genomics application: group individuals by genetic similarity

Genes

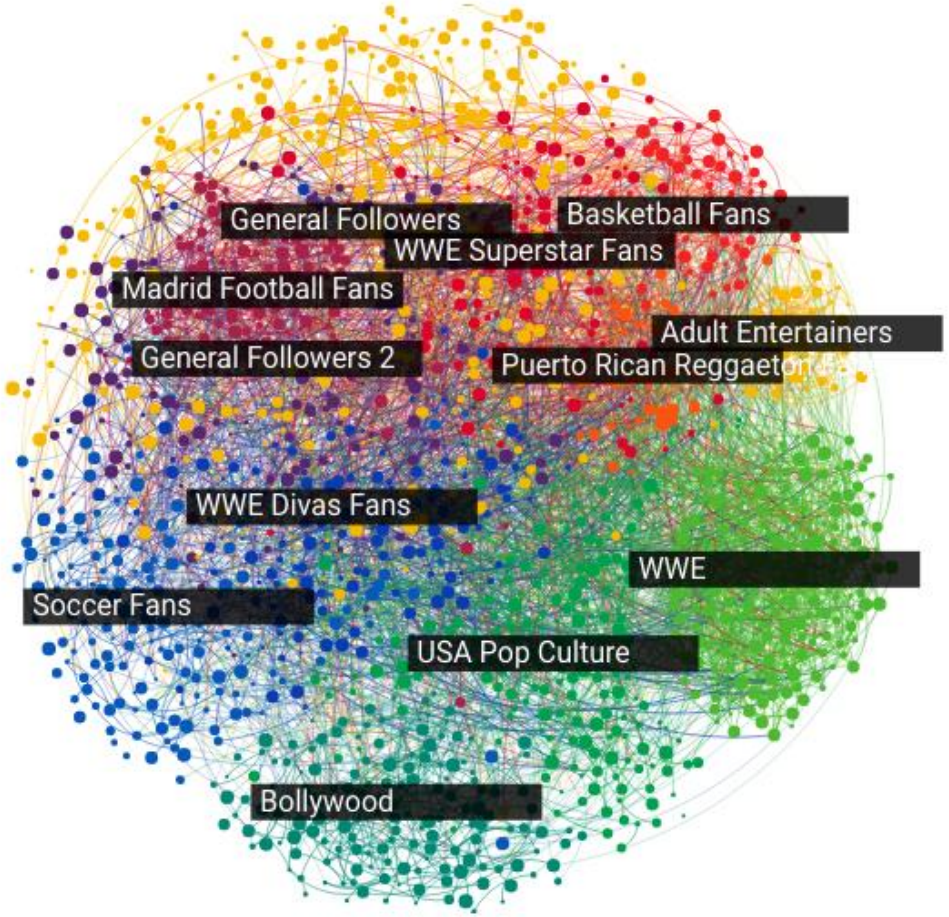


Individuals

[Source: Daphne Koller]

Data Mining

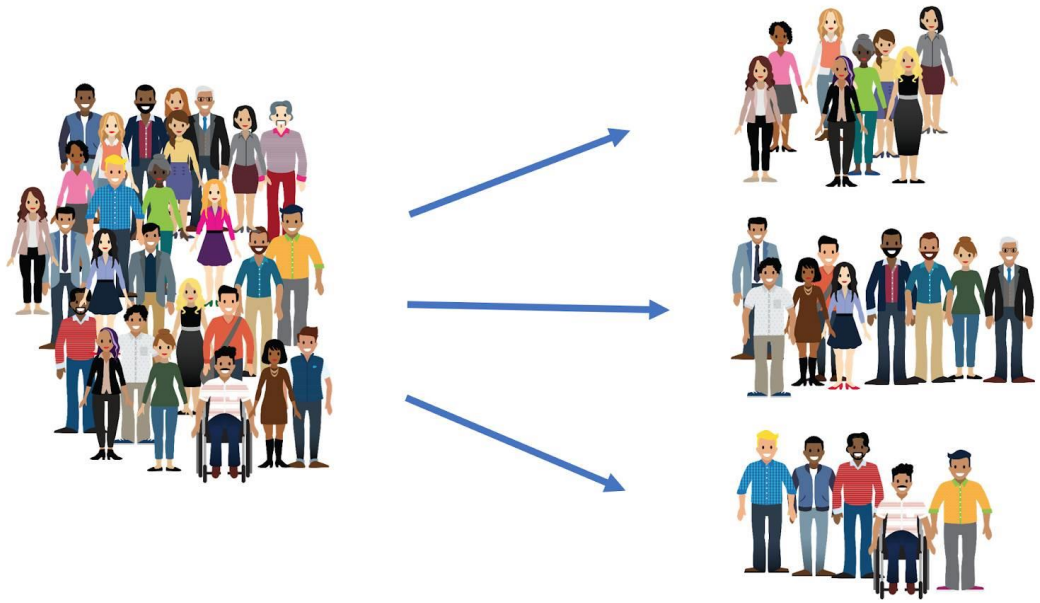
- Clustering is one of the most used data mining techniques.



https://www.affinio.com/blog/16-affinio-culture-series-the-wwe-universe/

Advertisement

- In marketing, segment customers according to their similarities and preferences



Other applications

- Clustering can be used in almost every field. For example, medicine, psychology, botany, sociology, biology, archeology, marketing, insurance, libraries, etc.

- In recent years, due to the rapid increase of online documents, text clustering becomes important.

K mean clustering

- The k-means clustering algorithm is an algorithm to cluster *n* objects based on data attributes into *k* partitions, where $k < n$.
- The algorithm group the data point based on attributes/features into K number of group.
- K is positive integer number.
- The grouping is done by minimizing the sum of squares of distances between data and the corresponding cluster centroid.

Steps:

1) Based on requirement, choose k (seeds) to be the initial centroids i.e., cluster centers (initially random selection)

2) Assign each data point to the closest centroid

3) Re-compute the centroids using the current cluster memberships. (by calculating the mean (average) of all the data points belonging to each cluster. This new centroid becomes the center of the cluster.)

4) If a convergence criterion is not met, go to 2).

Convergence criterion

1. no re-assignments or minimum re-assignment of data points to different clusters

2. no change or minimum change of centroids

3. minimum decrease in the sum of squared error (SSE)

$$SSE = \sum_{j=1}^{k} \sum_{\mathbf{x} \in C_j} dist(\mathbf{x}, \mathbf{m}_j)^2$$

C_j is the jth cluster, m_j is the centroid of cluster C_j (the mean vector of all the data points in C_j), and $dist(x, m_j)$ is the distance between data point x and centroid m_j.

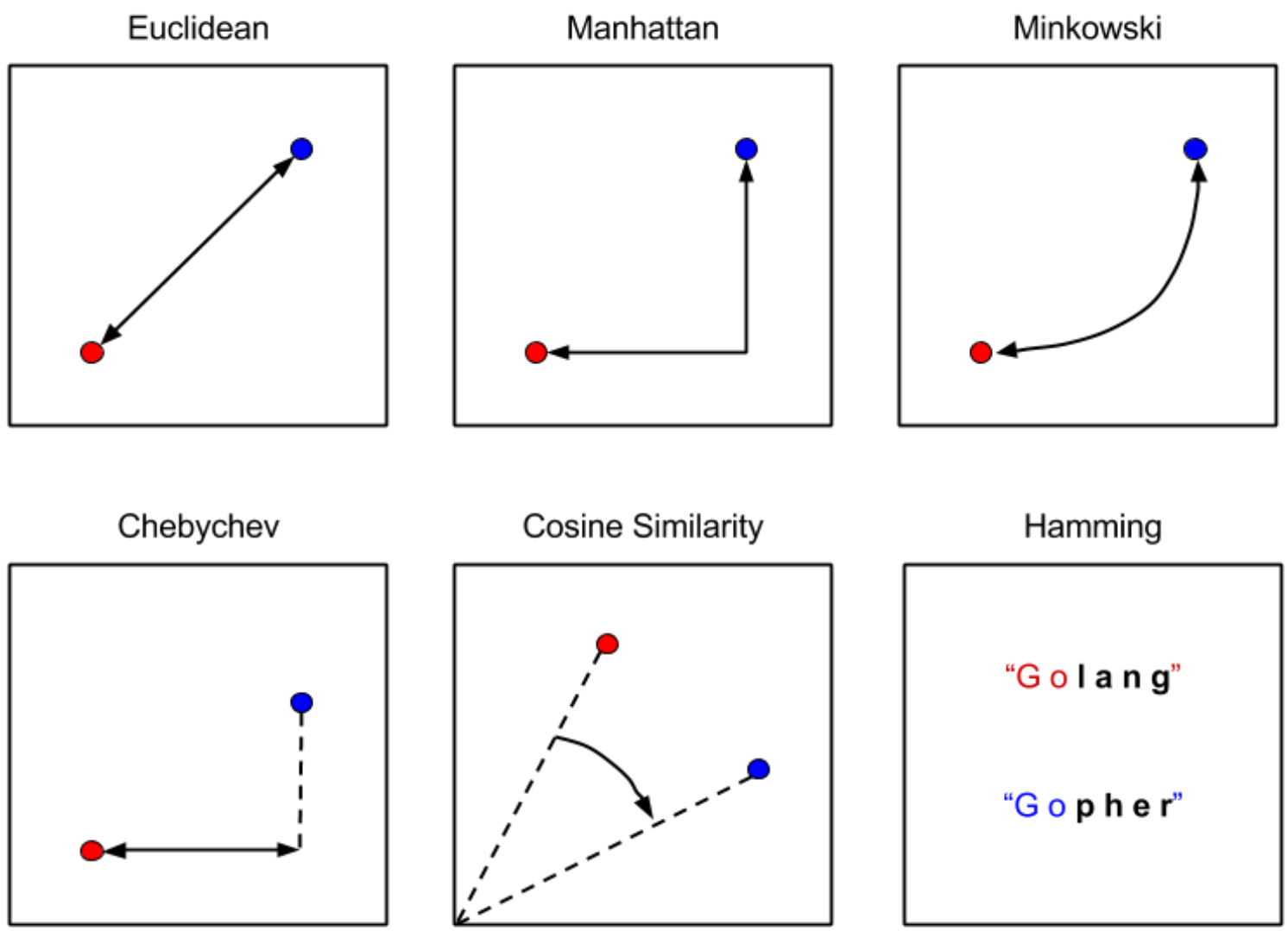
Types of distances

- Euclidean distance(2-norm $d(x,y)=\sqrt[2]{\sum_{i=1}^{p}|x_i-y_i|^2}$

- Manhattan distance(1-norm distance):

$$d(x,y)=\sum_{i=1}^{p}|x_i-y_i|$$

- Hamming distance measures the minimum number of substitutions required to change one member into another.



Slide by: https://subscription.packtpub.com/book/big_data_and_business_intelligence/9781785882104/6/ch06lvl1sec40/measuring-distance-or-similarity

K mean clustering (Flow diagram)

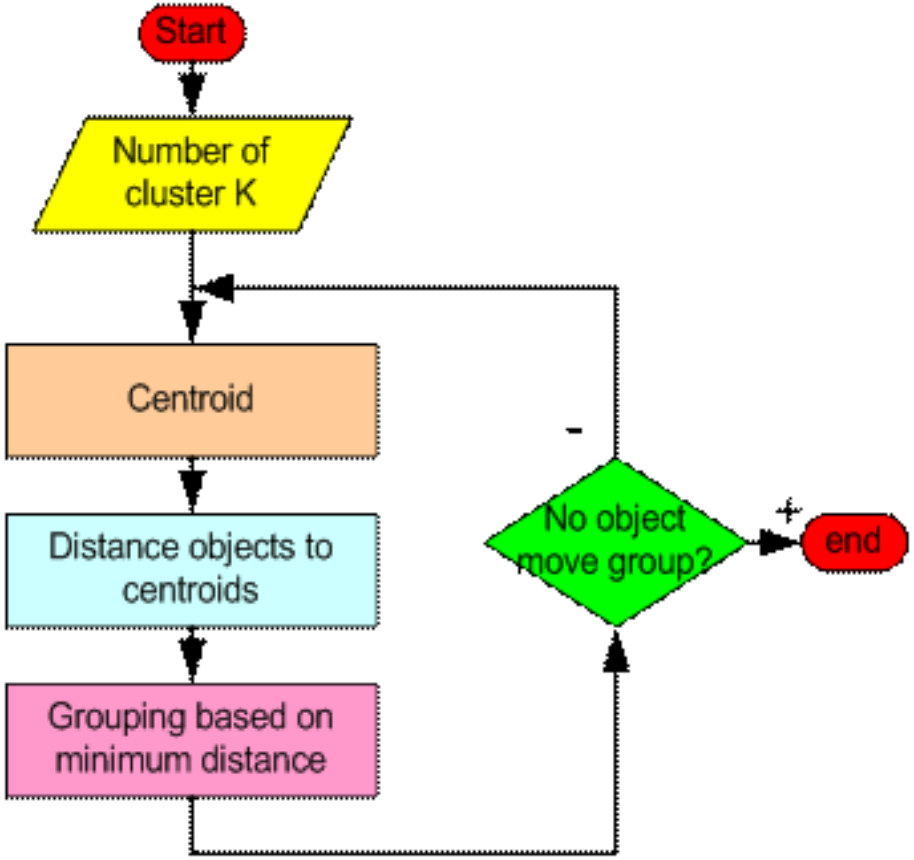


Figure by: https://www.researchgate.net/figure/Flow-Chart-for-K-Means-Clustering_fig5_308811921

Strengths of K-means

- Simple: easy to understand and to implement.
- K-means is considered a linear algorithm.
- K-means is the most popular clustering algorithm and works well in simple cases.

Weaknesses of K-means

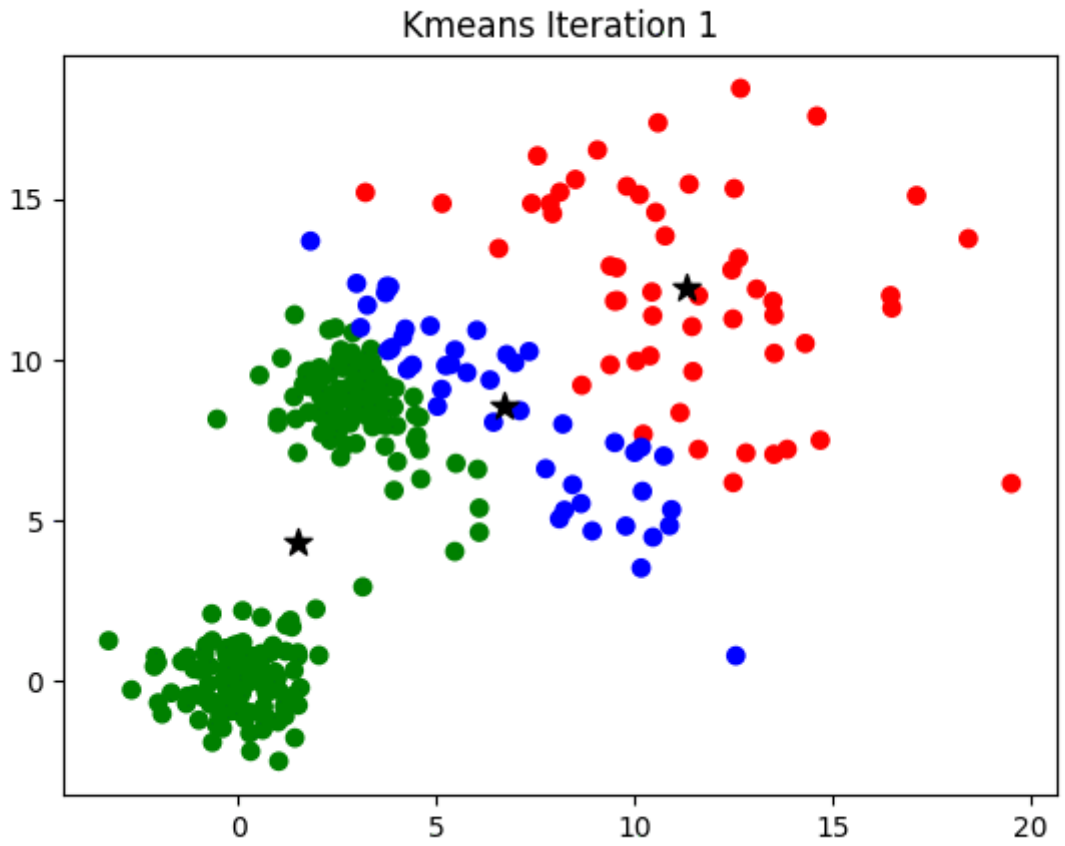
- The algorithm is only applicable if the mean is defined.

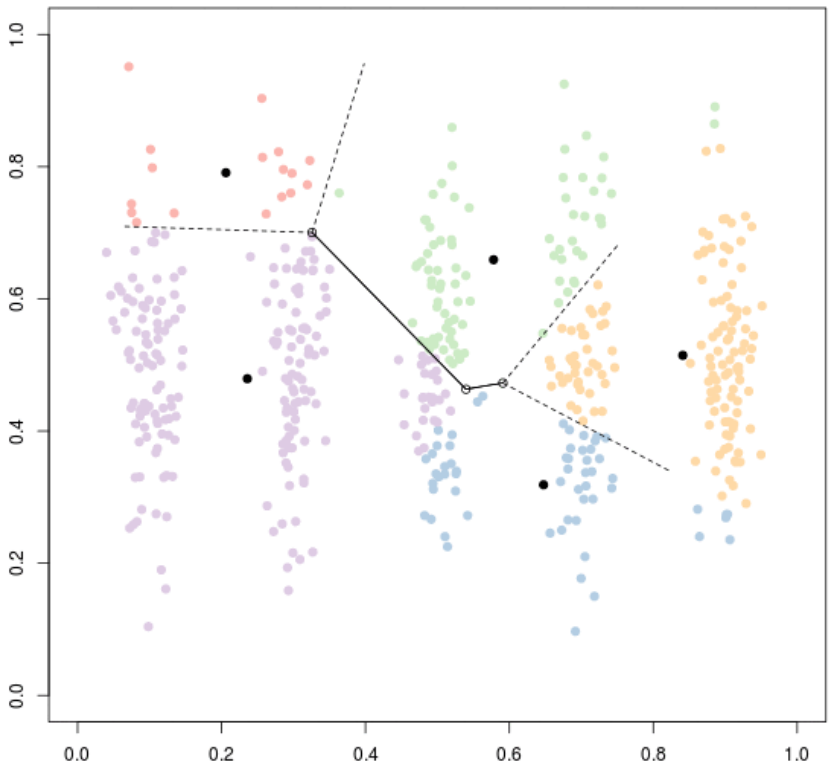
- The user needs to specify *k*.

- It terminates at a local optimum if SSE is used. The global optimum is hard to find due to complexity.

- The algorithm is sensitive to outliers
  - Outliers are data points that are very far away from other data points.
  - Outliers could be errors in the data recording or some special data points with very different values. (noisy data points)

Visualization





Optimal number of clusters

- The elbow method is a technique used to determine the optimal number of clusters, denoted as 'k', in k-means clustering. It helps in finding the "elbow point" in a plot of the cost or error of the clustering algorithm as a function of the number of clusters.

- The elbow method is a graphical technique that provides a way to estimate the optimal number of clusters in k-means clustering. However, it's important to note that the elbow point is not always very distinct, and the choice of 'k' can still require some domain knowledge and understanding of the specific dataset and problem at hand.

Optimal number of clusters (Steps)

1. Run K-means Clustering:

   Start by applying the k-means clustering algorithm to your dataset for a range of values of 'k', typically from 1 to a certain maximum value.

2. Calculate Cost for given k:

   For each value of 'k', calculate the cost of the clustering result. Cost is a measure of how far the data points within each cluster are from the centroid of that cluster. It is typically calculated as the **sum of squared distances** between data points and their centroid within each cluster.

3. Plot the Results:

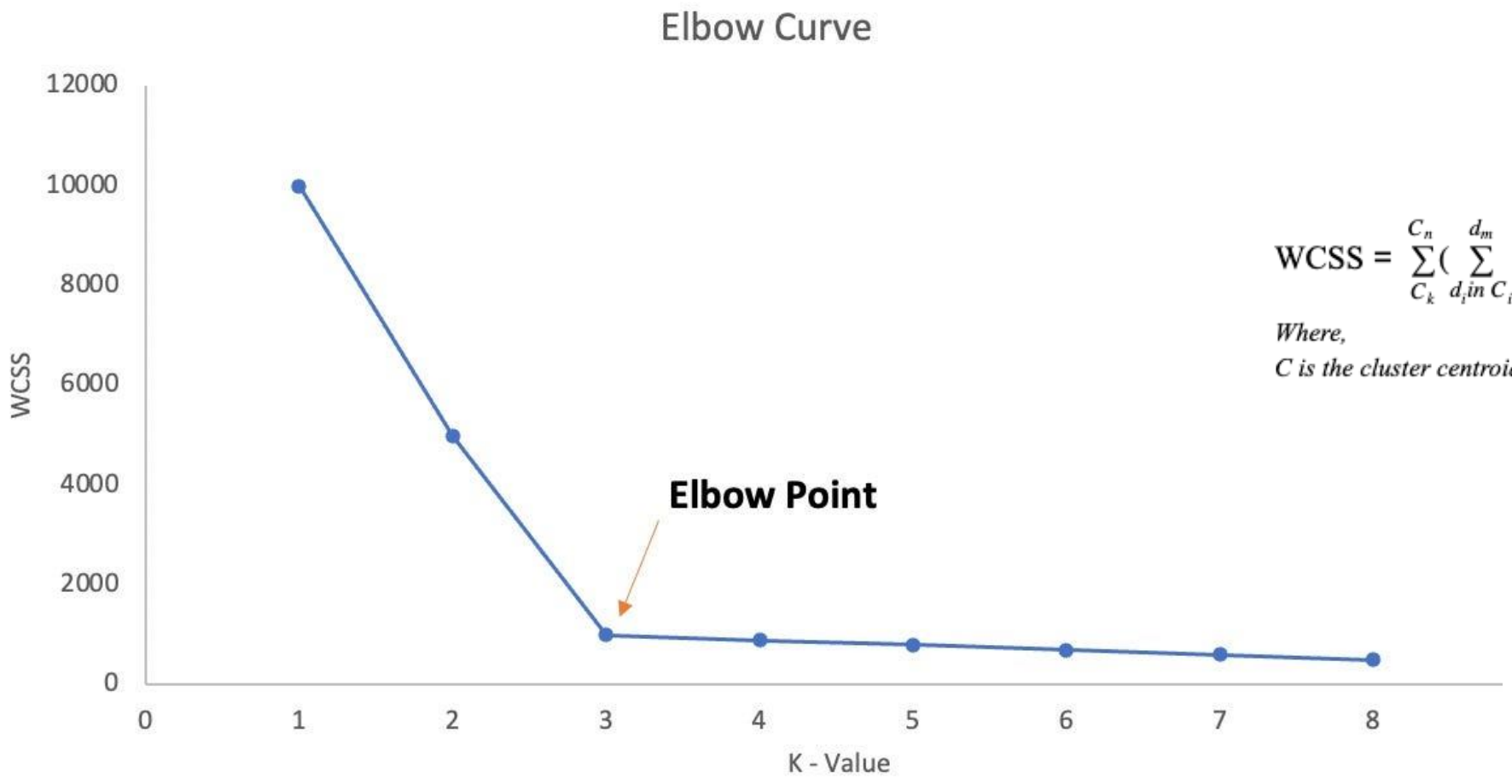
   Create a line plot with the number of clusters ('k') on the x-axis and the corresponding cost on the y-axis. This plot shows how the cost changes as 'k' increases.

4. Identify the Elbow Point:

   Look for the "elbow point" in the plot, which is the value of 'k' where the cost or inertia starts to level off or decrease at a slower rate. The elbow point is typically considered as the optimal number of clusters for k-means clustering.

5. Choose Optimal 'k':

   Based on the elbow point, you can choose the optimal number of clusters ('k') for your specific dataset. A lower value of 'k' may result in fewer clusters with higher intra-cluster similarity but may also result in more noise or misclassification. A higher value of 'k' may result in more clusters with lower intra-cluster similarity but may also overfit the data or lead to clusters with few data points.



$$\text{WCSS} = \sum_{C_k}^{C_n} \left(\sum_{d_i in\, C_i}^{d_m} distance(d_i, C_k)^2 \right)$$

Where,

C is the cluster centroids and d is the data point in each Cluster.

Types of Clustering

Partitional clustering

Partitional clustering is the simplest form of clustering that is used to classify data into groups based on their similarity.

Hierarchical clustering

Hierarchical clustering is the type of clustering that is used to produce a nested sequence of clusters, usually a tree like structure called dendrogram.

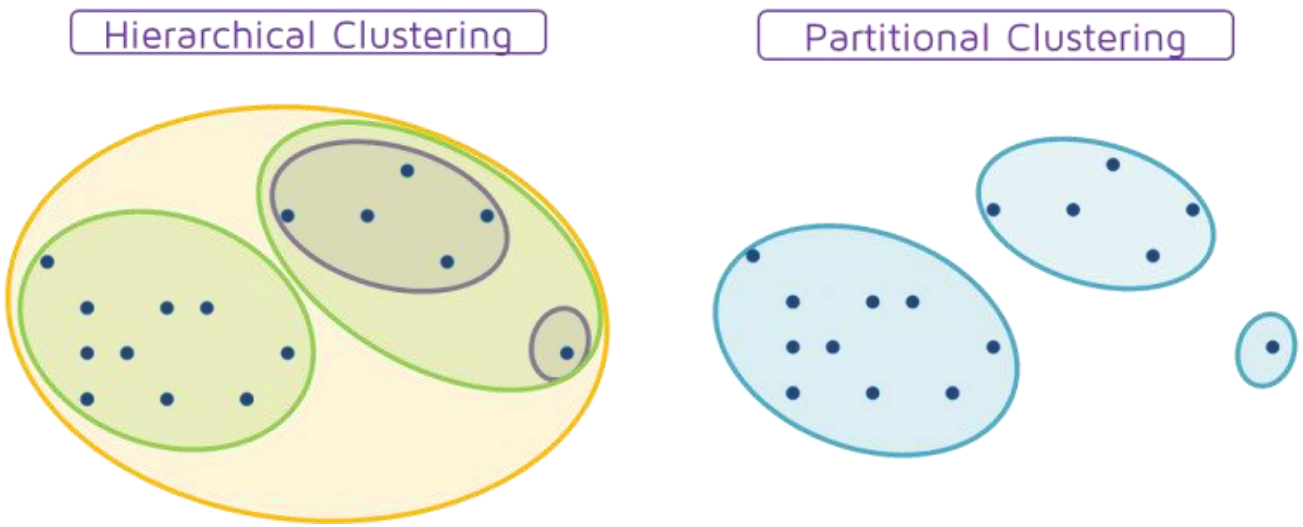
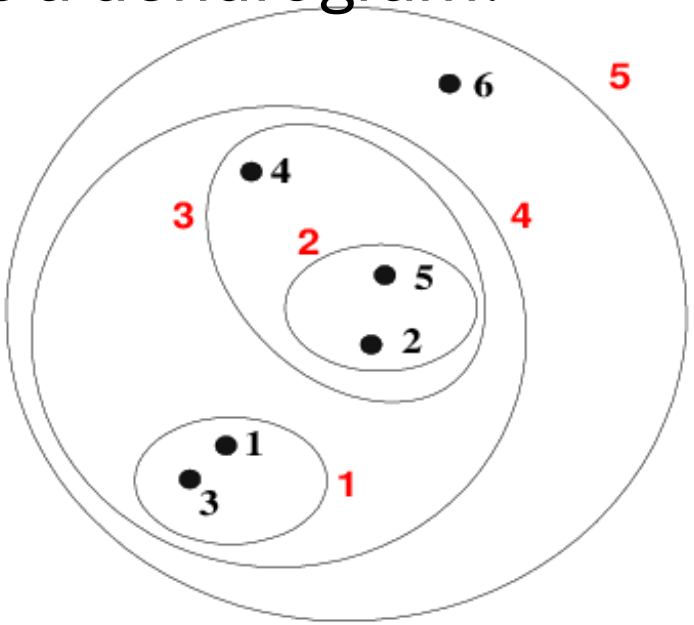


Figure courtesy: https://quantdare.com/hierarchical-clustering/hierarpartclustering/

Hierarchical clustering

- Produces a set of nested clusters organized a hierarchial tree.

- Can be visualized as a dendrogram.



Hierarchical clustering

- Two main types of hierarchical clustering

- Agglomerative
  - Start with points as individual clusters
  - At each step, merge the closes pair of clusters until only one cluster left

- Divisive
  - Start with one, all inclusive cluster
  - At each step, split a cluster until each cluster contains a point (or there are k clusters)

Internal vs External Clustering Evaluation Metrics

In clustering, **evaluation metrics are divided into two types based on what information they use**:

- **Internal metrics** is use *only the data and cluster assignments*
- **External metrics** is compare clustering results with *ground-truth labels*

Intuition

- **Internal metrics** answer:
  “How good is this clustering based only on the data itself?”

- **External metrics** answer:
  “How close is this clustering to the true labels?”

Internal Clustering Metrics

- Internal metrics evaluate clustering quality using only:
  - the data points
  - the distances between them
  - the cluster assignments

- No ground-truth labels are required

When are internal metrics used?

- Unsupervised learning (no labels available)
- Exploratory data analysis
- Comparing different values of *k* in k-means
- Real-world problems where true labels are unknown

What do internal metrics measure?

They typically measure **two properties**:

- **Cohesion** means data points in the same cluster should be close. In other words, similar data should be close to one another.

- **Separation** means data points in different clusters should be far apart. In other words, different data should be we far from one another in the feature space.

Clustering Metrics (Unsupervised AI)

1. Silhouette Score Measures how similar an object is to its own cluster (cohesion) compared to other clusters (separation).

- Range: **-1 to +1**
- Higher is better (close to +1 = well-separated, compact clusters; near 0 = overlapping; negative = possible misassignment).
- Very widely used because it's intuitive and works well for assessing cluster quality visually and numerically.

2. Davies-Bouldin Index (DB Index) Evaluates the average similarity between each cluster and its most similar one (ratio of within-cluster scatter to between-cluster separation).

- Range: **≥ 0** (lower is better; 0 would be perfect separation).
- Useful for comparing different numbers of clusters or algorithms; penalizes overlapping or poorly separated clusters.

Clustering Metrics (Unsupervised AI)

3. Calinski-Harabasz Index (Variance Ratio Criterion) Ratio of between-cluster dispersion to within-cluster dispersion.

- Range: **> 0** (higher is better).
- Especially good for compact, well-separated clusters (e.g., in k-means-like algorithms); often used in elbow-method extensions to choose the number of clusters.

4. Inertia (Within-Cluster Sum of Squares – WCSS) Sum of squared distances between each point and its cluster centroid.

- Range: **≥ 0** (lower is better).
- Primary metric for k-means; commonly plotted in the "elbow method" to find the optimal k. Note: it always decreases with more clusters, so it's not comparable across different algorithms.

Popular Metrics (when ground-truth labels are available for benchmarking)

- **Adjusted Rand Index (ARI)** Measures similarity between predicted clusters and true labels, adjusted for chance.
  - Range: **-1 to +1** (1 = perfect match, 0 = random, negative = worse than random).
  - One of the most robust and commonly reported external metrics.

- **Normalized Mutual Information (NMI)** Quantifies shared information between cluster assignments and true labels, normalized to [0,1].
  - Higher is better (1 = perfect agreement).

Quick Comparison Table

Metric	Type	Best Value	Main Strength	Common Use Case
Silhouette Score	Internal	Close to +1	Intuitive cohesion/separation balance	General quality check
Davies-Bouldin Index	Internal	Close to 0	Penalizes poor separation	Comparing clusterings
Calinski-Harabasz	Internal	As high as possible	Good for compact clusters	Choosing k in k-means variants
Inertia (WCSS)	Internal	As low as possible	Simple for centroid-based methods	Elbow method for k selection
Adjusted Rand Index	External	Close to +1	Chance-adjusted, robust	Benchmarking against labels
Normalized MI	External	Close to 1	Information-theoretic	When labels exist (research)

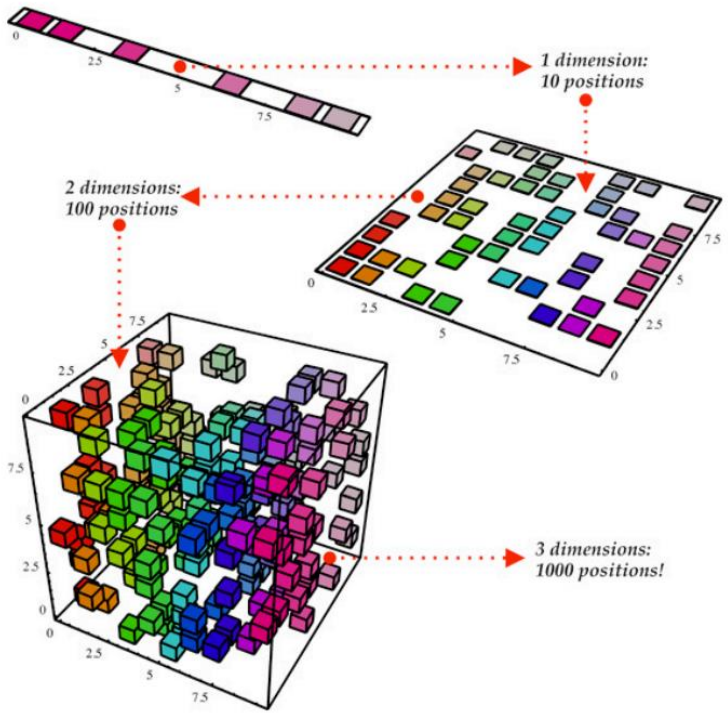
Dimensionality reductic

- Dimensionality reduction is a technique used in machine learning and data analysis to reduce the number of input variables or features in a dataset.

- The aim of dimensionality reduction is to simplify the data without losing important information.

- When datasets have a large number of features, they can be computationally expensive and can lead to overfitting, which reduces the model's ability to generalize to new data.

- Dimensionality reduction helps to overcome these problems by reducing the number of features in the dataset while retaining as much of the original information as possible.



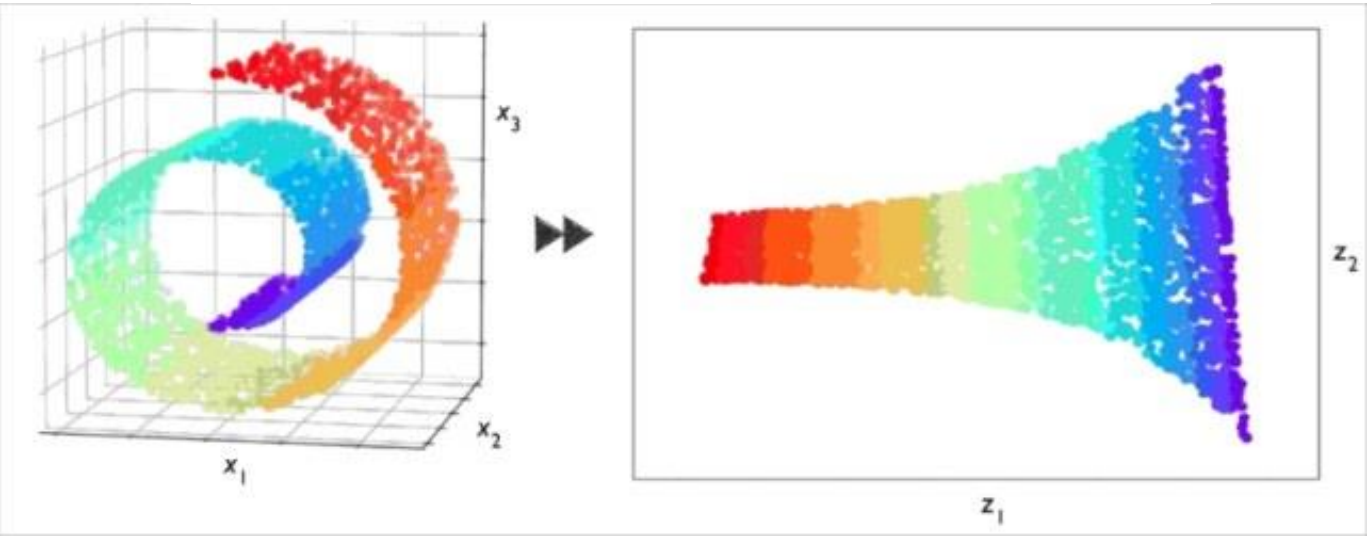


Figure courtesy: https://www.pinecone.io/learn/dimensionality-reduction/

Dimentionality Reduction (DR)

- Dimensionality reduction is the process of reducing the number of input variables or features in a dataset while preserving as much relevant information as possible.

- It converts high-dimensional data into a lower-dimensional form.

- It simplifies data without losing its essence.

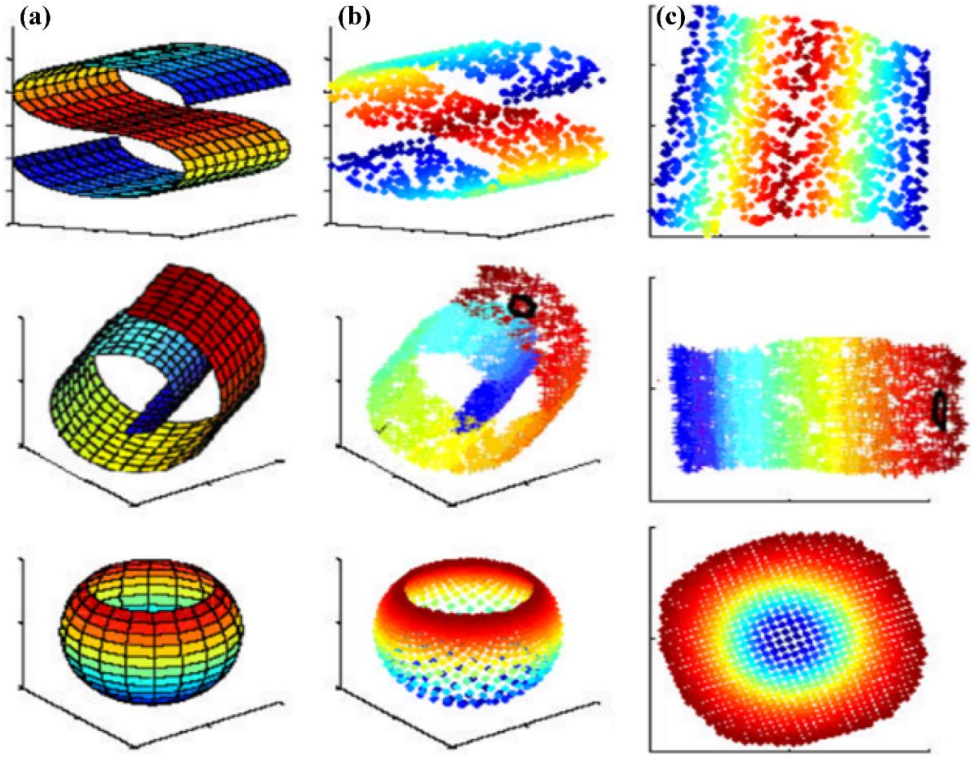


Figure courtesy: Jia, W., Sun, M., Lian, J., & Hou, S. (2022). Feature dimensionality reduction: a review. *Complex & Intelligent Systems*, *8*(3), 2663-2693.

Why do we need DR?

Overfitting Prevention

- High-dimensional data can lead to overfitting, where models learn noise instead of patterns.
- Reducing dimensions minimizes this risk.

Improved Model Performance

- Removes redundant and irrelevant features, leading to faster training and better predictions.

Enhanced Interpretability

- Simplifies datasets, making patterns easier to understand and visualize.

Reduced Computational Cost

- Lower dimensions reduce the time and resources needed for data processing and model training.

Applications of DR

Data Visualization

- Reduces high-dimensional data to 2D or 3D for easy visualization (e.g., clustering).

Preprocessing in Machine Learning

- Removes noise and redundant features before model training (e.g., in classification tasks).

Image Processing

- Compresses high-resolution images while retaining key features (e.g., facial recognition).

Genomics and Bioinformatics

- Reduces the complexity of genetic data for disease prediction and drug discovery.

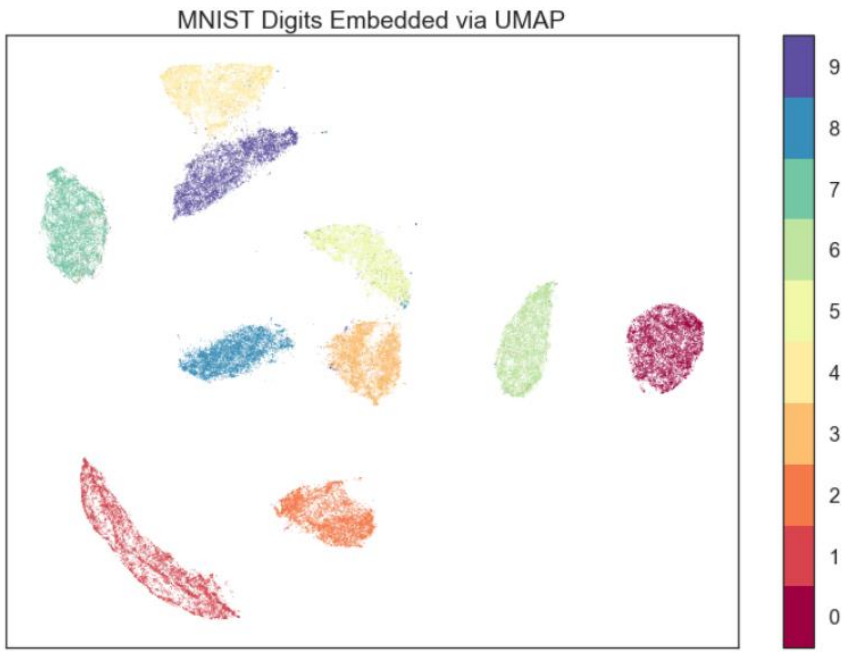


Figure courtesy: https://github.com/lmcinnes/umap

Types of DR

We can classify DR into three broad categories

1.Feature Selection:

Selecting a subset of the original features (e.g., variance thresholds, correlation thresholds).

2. Feature Extraction:

Transforming data into a new set of features (e.g., PCA, LDA).

3. Deep learning:

Using neural networks like auto-encoder for learning compressed representation of the data.

Methods of DR

Feature Selection

- **Variance Threshold**
- **Correlation Threshold**
- Recursive Feature Elimination (RFE)

Feature Extraction

- **Principal Component Analysis (PCA): Converts features into orthogonal components.**
- **Linear Discriminant Analysis (LDA): Maximizes class separability.**
- t-SNE and UMAP: For non-linear dimensionality reduction.

Deep learning

- **Neural network-based method for learning compressed representations of data.**

Variance Threshold

- Useful for eliminating constant or near-constant features.
- It is a filter based method.
- Applicable only on numerical features.
- Removes features with variance below a set threshold.
- It assumes higher variance may contain more useful information.
- The idea is to computer the variance of each feature (independently), and select the subset of features based on a user-specified threshold (α).
- This algorithm looks only at the features (X), not the desired outputs (Y).

Variance threshold

- Step 1: Let us consider the variance Threshold (α)>=1.9

- Step 2: Finding mean of the feature F1 and F2 respectively using m = 1/n$\sum_{i=1}^{n}(x_i)$

  Mean (F1): m1 =(5+4+3+6+2)/5 = 4

  Mean(F2): m2 =(1+4+5+3+4)/5 = 3.4

- Step 3: Finding the variance of F1 and F2 using vi = 1/n$\sum_{i=1}^{n}(x_i - m)^2$

  $V_1 = [(5-4)^2+(4-4)^2+(3-4)^2+(6-4)^2+(4-3)^2]/5 = 2$

  $V_2 = [(1-3.4)^2+ (4-3.4)^2+ (5-3.4)^2+ (3-3.4)^2+ (4-3.4)^2]/5 = 1.84$

- Step 4: Comparing the variance with our threshold variance (α) which was 1.9

  V_1 >= α (It can be chosen in our feature subset)

  V_2 >= α (It can not be chosen in our feature subset)

- Step 5: Feature F1 is chosen among the features F1 and F2.

F1	F2	Class
5	1	C1
4	4	C1
3	5	C2
6	3	C2
2	4	C1

Strength & Weakness

Strength

- Simplicity
- Speed
- Noise removal
- Interpretability

Weakness

- Correlation ignorance
- Scaling sensitivity
- Dependent variable independence
- Limited DR

Correlation Threshold

- Eliminates features that are highly correlated with others.
- The idea is to identify pairs of features with a correlation coefficient (e.g., Pearson's r) above a specified threshold and eliminate one feature from each pair.
- The method assumes that features with high correlation contribute similar information to the model, and thus one can be removed without significant loss of information.
- It assumes that a fixed threshold (e.g., $|r|>0.8$) is appropriate for identifying redundancy, regardless of the specific dataset or problem.

Correlation Threshold

- Step 1: Let us consider the variance Threshold (r)>0.9
- Step 2: Calculate the correlation matrix using Pearson correlation coefficient formula. For two feature X_i and X_j, the Pearson correlation coefficient r_{ij} is given by:

$$r_{ij} = \frac{\mathrm{Cov}(X_i, X_j)}{\sigma_{X_i} \cdot \sigma_{X_j}}$$

Where Cov(X_i,X_j) is the covariance between X_i and X_j

$$\mathrm{Cov}(X_i, X_j) = \frac{1}{n} \sum_{k=1}^{n} \left(X_{i,k} - \bar{X}_i\right) \left(X_{j,k} - \bar{X}_j\right)$$

n is the number of sample.

$X_{i,k}$, and $X_{j,k}$ are the k-th values of feature X_i and X_j

σ_{xi} and σ_{xj} are the standard deviation of X_i and X_j

$$\sigma_{X_i} = \sqrt{\frac{1}{n} \sum_{k=1}^{n} \left(X_{i,k} - \bar{X}_i\right)^2}$$

Correlation Threshold

- Step 3: Apply the correlation threshold ($|r|>0.9$)
  - X_1 and X_2: r = 0.995 (Remove X_2)
  - X_1 and X_3: r = 0.995 (Remove X_3)
  - X_1 and X_4: r = 0.9 (Keep both since r=0.9 is the threshold)

- Step 4: Retain Features X_1 and X_4 and discard the others.

Feature 1 (X_1)	Feature 2 (X_2)	Feature 3 (X_3)	Feature 4 (X_4)
1.0	1.1	2.0	0.5
2.0	2.1	4.0	1.0
3.0	3.1	6.0	1.5
4.0	4.1	8.0	2.0
5.0	5.1	10.0	2.5

$$\text{Correlation Matrix} = \begin{bmatrix} 1.0 & 0.995 & 0.995 & 0.9 \\ 0.995 & 1.0 & 0.995 & 0.91 \\ 0.995 & 0.995 & 1.0 & 0.92 \\ 0.9 & 0.91 & 0.92 & 1.0 \end{bmatrix}$$

Strength & Weakness

Strength

- Reduced redundancy
- Scalable
- Flexibility
- Interpretability

Weakness

- Ignores Non-linear Relationships
- Threshold Sensitivity
- Feature Importance Overlooked
- Feature sensitivity

Principal component analysis

- A feature extraction method that transforms data into orthogonal components.

- Each component captures the maximum variance in the data.

- **Advantages:**
  - Reduces redundancy by creating uncorrelated components.
  - Retains most of the data's variability in fewer dimensions.

- **Assumptions**
  - Linear relations
  - Variance and information
  - Feature scaling matters

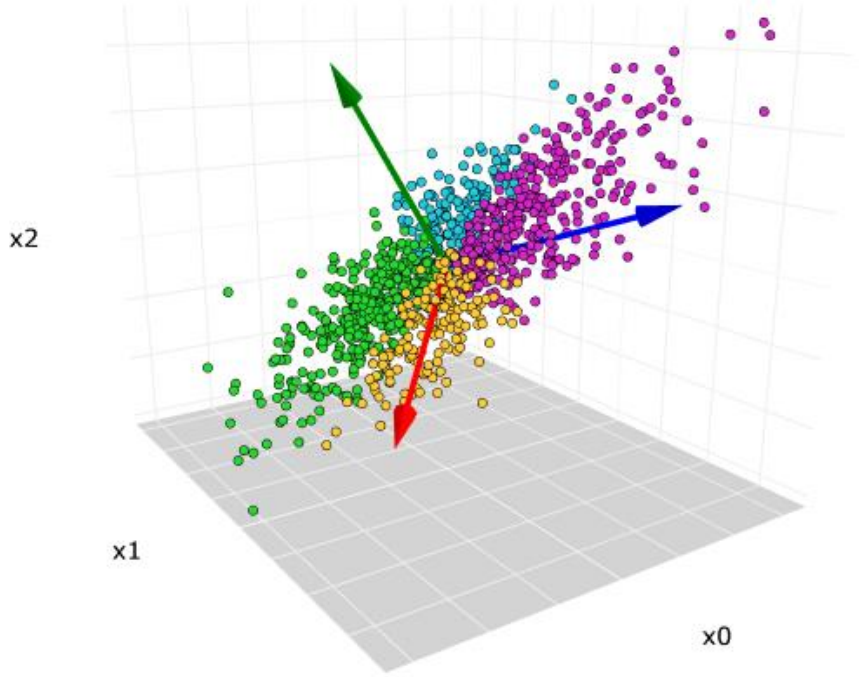


Figure courtesy: https://medium.com/@ilyurek/principal-component-analysis-pca-a-practical-guide-58dea99dd93

Step for PCA computation

- Standardization of the data
- Computing the covariance matrix
- Calculating the eigenvectors and eigenvalues ($A\mathbf{v} = \lambda\mathbf{v}$) -> $(A- \lambda I)\mathbf{v} = 0$
- Computing the Principal Components (Sort eigen values and vectors)

Eigen vectors and covariance matrix

- The idea behind eigenvectors is to use the Covariance matrix to understand where in the data there is the most amount of variance.

- Since more variance in the data denotes more information about the data, eigenvectors are used to identify and compute Principal Components.

- After calculating the eigenvectors and eigenvalues, we order them in the descending order, where the eigenvector with the highest eigenvalue is the most significant and thus forms the first principal component. The principal components of lesser significances can thus be removed in order to reduce the dimensions of the data.

Image compression

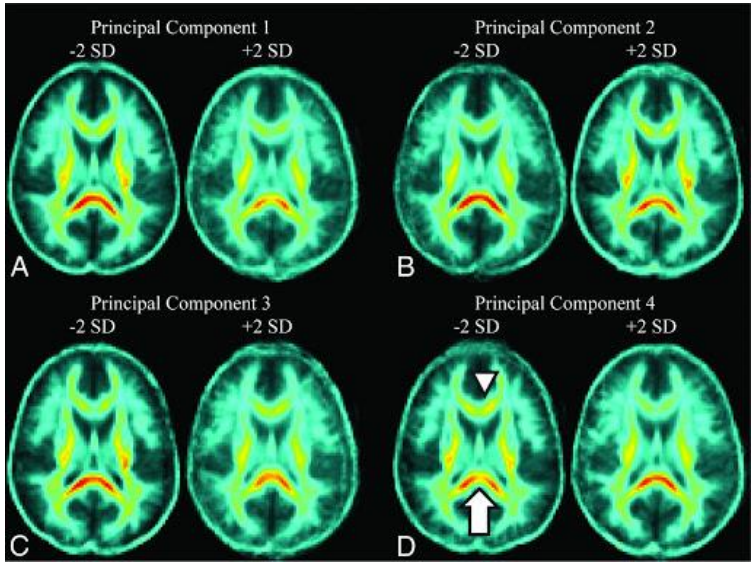
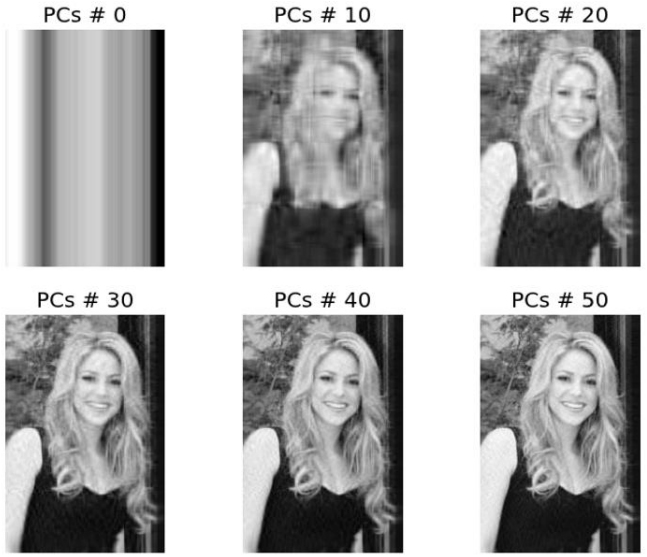


Image compression



Face recognition

Figure courtesy: https://github.com/SilvesterHsu/ORLFaceRecognition-PCA

Strength & Weakness

Strength

- Reduced redundancy
- Versatility
- Noise reduction
- Visualization

Weakness

- Linear Assumption
- Loss of Interpretability
- Variance-Based Selection May Miss Important Features
- Sensitive to Scaling

Linear Discriminant Analysis (LDA)

- LDA is a dimensionality reduction technique that projects the data onto a lower-dimensional space to maximize class separability.

- Unlike PCA, which focuses on preserving variance, LDA focuses on maximizing the distance between class means while minimizing the scatter within each class.

- The resulting components are linear combinations of the original features, designed to improve classification tasks.

- A supervised method that reduces dimensions while maximizing class separability.

- Advantages:
  - Effective for classification tasks.
  - Focuses on preserving class-specific information.
- Assumption:
  - Assumes Gaussian distribution and equal class covariances.
  - LDA assumes that classes can be separated linearly in the transformed space.
  - LDA assumes that all classes have the same covariance matrix.
  - Limited to labeled datasets.

Steps

- Compute class means
- Computer the scatter Matrices (S_W)

$$S_W = \sum_c \sum_{x \in \text{class } c} (x - \bar{x}_c)(x - \bar{x}_c)^T$$

- Computer Between Class Scatter Matrix (S_B)

$$S_B = n_0(\bar{x}_0 - \bar{x})(\bar{x}_0 - \bar{x})^T + n_1(\bar{x}_1 - \bar{x})(\bar{x}_1 - \bar{x})^T$$

- Computer LDA transformation

Strength & Weakness

Strength

- Class seperability
- Efficient (linear classes)
- Interpretable
- Simple implementation

Weakness

- Assumption of normality.
- Not Effective for Non-Linear Boundaries
- Class Imbalance Sensitivity
- Feature Scaling

Deep learning based

Auto encoder

- Neural network-based unsupervised learning technique.
- Encodes data into a compressed representation and reconstructs it.

Advantages:

- Captures complex, non-linear relationships.
- Can handle very high-dimensional data.

Limitations:

- Requires significant computational resources.
- Needs careful tuning to avoid overfitting.

Auto encoder

An autoencoder is a neural network that learns a compressed representation of data.
It consists of:

1. Encoder: Compresses input data into a lower-dimensional latent space.
2. Decoder: Reconstructs the original data from the latent representation.

- Key Features:
  - Captures non-linear relationships between features.
  - Works well for high-dimensional and complex data (e.g., images, text).
  - Trained using unsupervised learning to minimize reconstruction error.
- Assumptions:
  - The reduced representation should retain the most relevant information for reconstruction.
  - Sufficient data is required to train the network effectively.

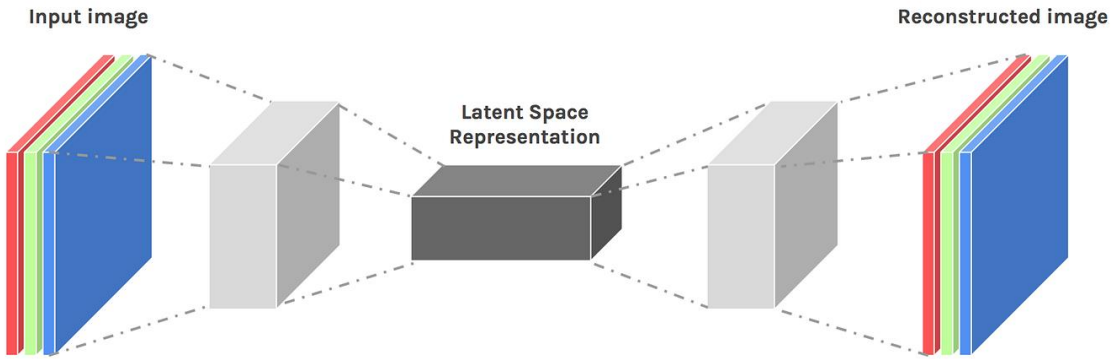


Figure Courtesy: https://towardsdatascience.com/autoencoders-bits-and-bytes-of-deep-learning-eaba376f23ad

Strength & Weakness

Strength

- Handles non-linear dimensionality reduction (unlike PCA or LDA).
- Flexible: Architecture can be customized for different types of data.
- Can handle noisy data through denoising autoencoders.

Weakness

- Computationally intensive compared to PCA/LDA
- Requires careful tuning of hyperparamerter and architecture
- May overfit with insuffience date and improper regularation

GAN

Generative Adversarial Networks (GANs) can be adapted for dimensionality reduction by learning a lower-dimensional representation of data in the latent space.

- Consists of two components:
  - Generator: Maps a low-dimensional latent vector to high-dimensional data.
  - Discriminator: Distinguishes between real and generated data.

- The latent space in the generator can serve as the reduced-dimensional representation.

- **Key Features:**
  - Captures complex, non-linear relationships in the data.
  - Suitable for high-dimensional data (e.g., images, audio).
  - Preserves important generative information in the latent space.
- **Assumptions:**
  - Data can be effectively represented in a lower-dimensional latent space.
  - The GAN is trained successfully (avoiding issues like mode collapse).

Strength & Weakness

Strength

- Generates high-quality synthetic data in addition to dimensionality reduction.
- Latent space is interpretable and can capture meaningful features.
- Handles non-linear transformations better than linear methods like PCA.

Weakness

- Training GANs is computationally expensive and sensitive to hyperparameters.
- Prone to instability (e.g., mode collapse).
- Requires large datasets for effective training.

Summary

Method	Supervised/Unsupervised	Linear/Non-Linear	Scalability	Interpretability	Best Use Case
Variance Threshold	Unsupervised	Linear	High	High	Initial noise filtering
Correlation Threshold	Unsupervised	Linear	High	Medium	Redundancy reduction
PCA	Unsupervised	Linear	Medium	Low	High-dimensional data simplification
LDA	Supervised	Linear	Medium	Medium	Classification tasks
Autoencoder	Unsupervised	Non-Linear	Medium-High	Low	Complex feature extraction
GAN	Unsupervised	Non-Linear	Low-Medium	Low	Synthetic data generation

Take away

- Method Selection Depends on the Goal: Consider interpretability, scalability, and the nature of your data.
- Combine Methods: Use simple methods like Variance or Correlation Threshold for preprocessing and advanced techniques like PCA, LDA, or Autoencoders for fine-tuning.
- Balance Trade-offs: Between dimensionality reduction, information retention, and model performance.
- Dimensionality reduction is not just a preprocessing step—it's a pathway to uncovering hidden insights and improving model efficiency.

Challenges

Information Loss:

Reducing dimensions can lead to the loss of important data.

Scalability:

Handling very large datasets efficiently.

Algorithm Selection:

Choosing the right method for the task (linear vs. non-linear).

Anomaly detection

- Cyber crime
- Transaction frauds
- ID theft.

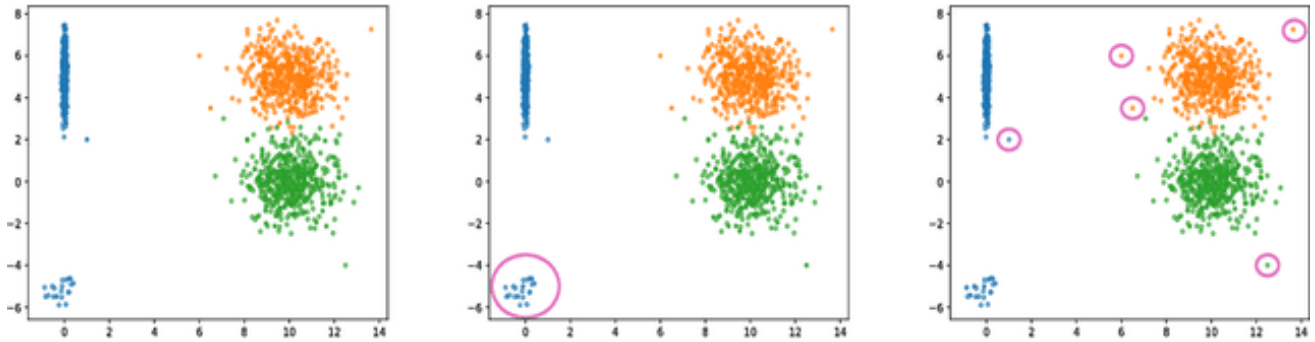
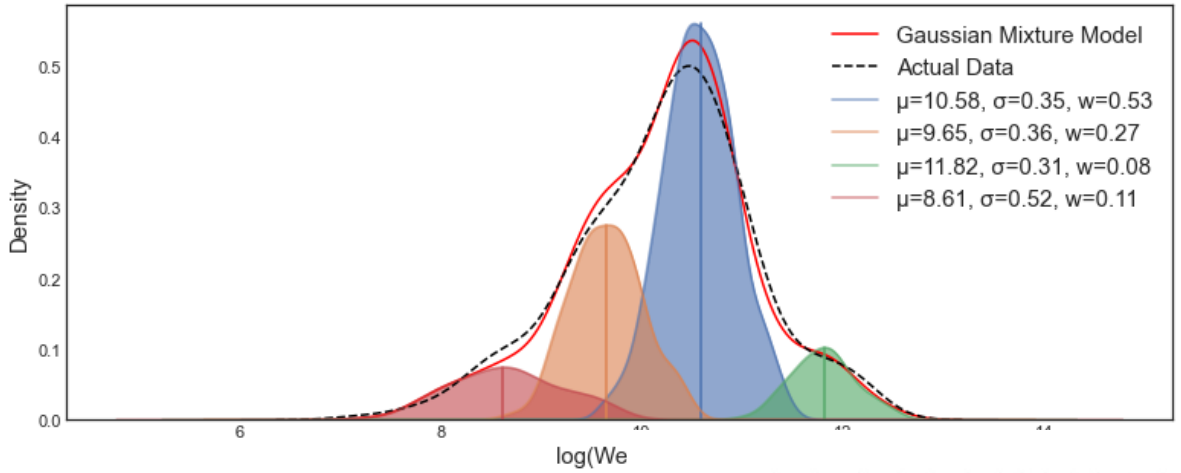


Figure courtesy: https://www.catalyzex.com/paper/arxiv:1805.09411

Unsupervised learning algorithms

- Clustering
- Gaussian mixture model
- Principal component analysis (PCA)
- K singular value decomposition (KSVD)
- Independent component analysis.



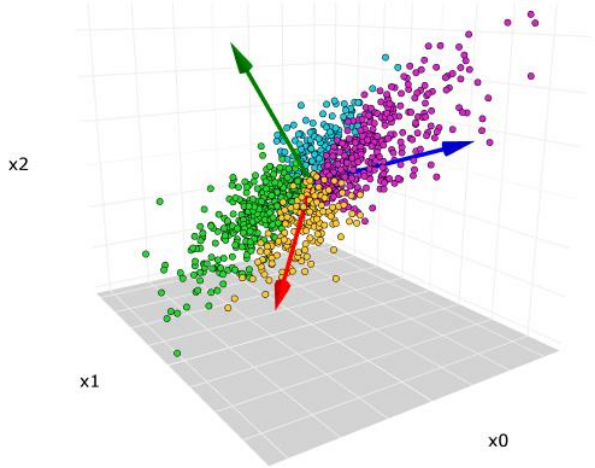


Figure courtesy: https://stats.stackexchange.com/questions/517652/how-to-evaluate-the-loss-on-a-gaussian-mixture-model

Slides adapted from:

- Pattern Recognition and Machine Learning Book by Christopher Bishop
- Kevin Murphy, UBC Machine learning course
- Mubarak Shah, UCF Computer Vision course
- Farid Melgani, UoT Pattern Recognition course
- Pattern Classification Book by Duda and Hart

References

- Pattern Classification by Duda and Hart.
- Pattern recognition and machine learning by Christopher Bishap.
- Digital Image Processing and Analysis-by B.Chanda and D.Dutta Majumdar.
- J. A. Hartigan (1975) "Clustering Algorithms". Wiley.
- J. A. Hartigan and M. A. Wong (1979) "A K-Means Clustering Algorithm", Applied Statistics, Vol. 28, No. 1, p100-108.